



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

STEGANOGRAFIA DIGITALĂ: TEHNICI, ALGORITMI ȘI APLICAȚII

Absolvent

Miclea Alexandru

Coordonator științific

Conf. Dr. Ing. Rusu Cristian

București, iunie 2025

Rezumat

Steganografia reprezintă un mijloc prin care folosind un mediu de transportare, putem ascunde un mesaj într-o manieră în care o persoană, dacă nu suspectează prezența mesajului, nu ar avea cum să își dea seama de existența acestuia. Steganografia digitală se referă la studiul metodelor de ascundere a mesajelor în medii digitale, precum imagini, videoclipuri sau conținut audio. În lucrarea de față sunt prezentate 4 modele de ascundere a mesajelor secrete în imagini și fișiere audio, metode de steganaliză pentru detectarea prezenței mesajelor secrete în aceste medii și algoritmi care să faciliteze ascunderea și extragerea mesajelor.

Abstract

Steganography represents a means through which, using a cover work, we can hide a message in such a way that it is imperceptible to a person, if they aren't aware of its presence. Digital steganography refers to the study of models for embedding hidden messages in digital works such as images, videos or audio content. In this work we present 4 models used for secret message embedding in images and audio files, methods to detect the presence of hidden content using steganalysis and algorithms that help with the embedding and extracting of messages.

Cuprins

1	Introducere	5
1.1	Despre steganografie	5
1.2	Scopul lucrării	5
1.3	Motivarea științifică și personală	5
1.4	Contribuția personală	6
1.5	Structura lucrării	6
2	Preliminarii	7
2.1	Mediu, robustețe, ascundere, imperceptibilitate	7
2.2	Steganaliza	8
2.3	Transformata cosinus discretă	8
2.4	Transformata Fourier discretă	9
2.5	Interpretarea unui semnal audio digital	9
3	Implementări	10
3.1	Despre implementare	10
3.2	Metrici de măsurare	10
3.2.1	Eroarea pătratică medie	10
3.2.2	Raportul semnal maxim-zgomot	11
3.2.3	Indicele de similaritate structurală mediu	11
3.3	Modele de steganografie	12
3.3.1	Metoda bitului cel mai puțin semnificativ	12
3.3.2	Diferențierea valorilor pixelilor	13
3.3.3	Transformata cosinus discretă	14
3.3.4	Codarea de fază	15
3.4	Steganaliză	16
3.4.1	Inspecție vizuală	16
3.4.2	Metode statistice	18
3.4.3	Clasificator bazat pe rețele neurale	19
3.5	Codificare Huffman - Crearea mesajului de ascuns	19

4	Rezultate	21
4.1	Scoruri obținute	21
4.2	Scoruri folosind metode statistice	22
4.3	Matrici de confuzie pentru rețeaua neurală	22
5	Concluzii	23
5.1	Direcții viitoare de cercetare	23
	Bibliografie	24

Capitolul 1

Introducere

1.1 Despre steganografie

Termenul de steganografie provine din cuvântul grec steganographia (steganos = ascuns, graphia = scriere). Steganografia digitală a înregistrat o dezvoltare ca subiect de cercetare la începutul anilor '90, fiind utilizată pentru protejarea drepturilor de autor, evitarea cenzurii și comunicarea secretă, printre altele. Domeniile conexe cu steganografia sunt criptografia și filigranarea digitală. Spre deosebire de criptografie, unde un mesaj este cifrat folosind un algoritm și o cheie, iar obiectivul principal este confidențialitatea sau filigranarea digitală, care urmărește garantarea integrității unui mediu într-un mod mai mult sau mai puțin secret, steganografia are ca scop ascunderea unui mesaj în mediu, într-un mod imperceptibil.

1.2 Scopul lucrării

Scopul lucrării este de a prezenta rezultatele obținute din testarea mai multor modele de steganografie și de a identifica cel mai bun model din punct de vedere al gradului de imperceptibilitate, al capacității de ascundere sau al timpului luat pentru ascunderea și extragerea unui mesaj. De asemenea, sunt explorate modurile de folosire a steganalizei pentru a detecta prezența mesajelor, ținând cont de eficiența computațională și de gradul de implicare al unei persoane în proces, alături de eficiența lor pentru modelele propuse.

1.3 Motivarea științifică și personală

Steganografia reprezintă un domeniu de interes academic, relevanța sa fiind dată de nivelul de conținut digital care este produs și publicat zilnic pe internet. Există multiple exemple în literatura de specialitate și în presă ce prezintă moduri în care a fost folosită în trecut steganografia de către actori statali sau persoane rău intenționate. Câteva exem-

ple relevante sunt: folosirea imprimantelor¹ pentru a ascunde informații despre data și aparatul folosit pentru imprimare, folosirea de către servicii secrete pentru ascunderea și trimiterea informațiilor în mod imperceptibil pe internet², folosirea în securitatea cibernetică drept mijloc de a transporta conținut malițios, îngreunând detecția prin metode convenționale de securitate³ și spionaj industrial⁴. Într-un articol publicat în 2018 de către compania Doctor Web⁵ s-au prezentat rezultatele unei investigații din care reiese folosirea steganografiei pentru transportarea de conținut malițios în imaginile folosite de aplicații disponibile pe Google Play, unele fiind instalate de peste 4 milioane de ori.

1.4 Contribuția personală

Contribuția mea personală constă în implementarea modelelor de steganografie, a tehnicilor de steganaliză, a metricilor folosite și al modului de construire al mesajului ce urmează să fie ascuns. Folosind implementările am experimentat ascunderea și extragerea diferitelor forme de conținut digital. În urma experimentării am notat rezultatele obținute de fiecare model, capacitatea fiecărei tehnici de steganaliză de a detecta conținutul ascuns, avantajele și dezavantajele aduse de fiecare în parte. La baza acestor implementări a stat cercetarea în domeniu, sursele fiind alese din literatura de specialitate și din articole disponibile online.

1.5 Structura lucrării

Lucrarea este structurată în următoarele capitole:

1. **Introducere:** Prezintă informații despre steganografie, relevanța domeniului, motivarea și contribuția personală
2. **Preliminarii:** Definește termenii folosiți în steganografie, steganaliza, concepte matematice
3. **Implementări:** Oferă detalii de implementare pentru tehnicile de steganaliză, modele și metrici
4. **Rezultate:** Prezintă rezultatele obținute de către modele și tehnicile de steganaliză
5. **Concluzii:** Rezumă rezultatele și oferă concluzii despre viabilitatea modelelor

¹https://en.wikipedia.org/wiki/Printer_tracking_dots

²<https://www.darkreading.com/cyber-risk/busted-alleged-russian-spies-used-steganography-to-conceal>

³<https://builtin.com/articles/steganography>

⁴<https://www.bbc.com/news/world-asia-china-64206950>

⁵<https://news.drweb.com/show/?i=11685>

Capitolul 2

Preliminarii

2.1 Mediu, robustețe, ascundere, imperceptibilitate

Denumim mediu acele imagini, fișiere audio, video-uri în care ascundem un mesaj. Obiectivul steganografiei digitale este de a echilibra capacitatea de ascundere (cât mesaj putem ascunde într-un mediu dat) cu imperceptibilitatea și robustețea.

Prin ascundere ne referim la procesul prin care un mesaj (în mod tipic în format binar) este introdus într-un mediu digital. Independent de dimensiunea mediului în care alegem să ne ascundem mesajul, modelele lasă loc unui spațiu limitat în care să se facă ascunderea. Cu cât mărim spațiul în care poate fi ascuns un mesaj, cu atât devine mai greu de ascuns prezența unei modificări. Capacitatea de ascundere se raportează la dimensiunea mediului folosit, o metrică folosită în acest sens fiind *numărul de biți per pixel* (Bits Per Pixel).

Imperceptibilitatea reprezintă capacitatea unui model de a nu dezvălui faptul că un mesaj a fost ascuns. Un model este considerat ca oferind imperceptibilitate dacă prezența unui mesaj poate fi mascată împotriva atacurilor de steganaliză. Putem calcula matematic gradul de imperceptibilitate pe care un model îl oferă raportat la mediul original, folosindu-ne de metrici precum: *Eroarea pătratică medie*, *Raportul semnal maxim-zgomot*, *Indicele de similaritate structurală mediu*.

Robustețea unui model este dată de capacitatea unui mediu în care a fost ascuns conținut de a rezista unor ulterioare modificări (atacuri cu zgomot, compresie) fără a afecta recuperabilitatea mesajului ascuns. Este important de luat în calcul atunci când linia de comunicare pe care se pregătește a fi transmis mesajul stego nu este stabilă sau se cunoaște prezența unui potențial atacator activ.

Alegerea mediului în care se face ascunderea este de asemenea un lucru de luat în calcul. În codul aferent lucrării am folosit mai multe imagini de test folosite în literatură, însă pentru un grad mai mare de imperceptibilitate se recomandă folosirea mediilor noi, despre care să nu existe indicii că ar fi fost folosite în prealabil.

2.2 Steganaliza

Prin steganaliză înțelegem seria de proceduri pe care le putem aplica pentru a detecta prezența conținutului ascuns într-un mediu aleator. Câteva exemple de moduri prin care se poate face steganaliză pentru un conținut digital sunt: inspecție vizuală, metode statistice (Testul Chi-pătrat, divergența dintre două distribuții de probabilități) sau metode bazate pe inteligență artificială.

2.3 Transformata cosinus discretă

Transformata cosinus discretă este folosită în compresia jpeg a imaginilor digitale. Fiecare imagine este împărțită în regiuni de dimensiune 8×8 , convertită din formatul de culoare RGB în YCbCr, iar pentru canalul culoare Y (care denotă luminozitatea unui pixel în reprezentarea imaginii) descompunem zona în sume de componente cosinusoidale 2-dimensionale. În figura următoare putem vedea cele 64 de componente 2D folosite. Pentru fiecare componentă vom obține o valoare întreagă, reprezentând "cât de mult" din componenta respectivă apare în zona aleasă.

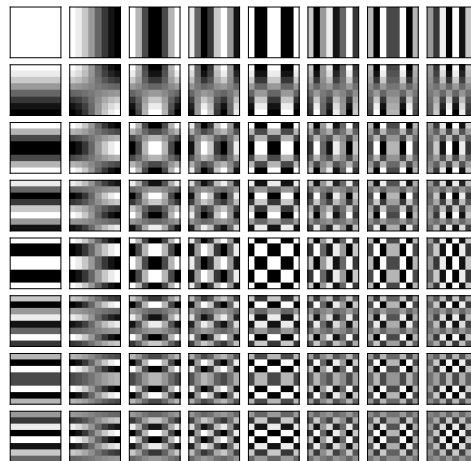


Figura 2.1: Componentele de frecvență 2D folosite de către transformata cosinus discretă

Formula DCT-II, folosită în cod, este:

$$X_{x,y} = \frac{1}{\sqrt{MN}} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) \cos \frac{(2x+1)m\pi}{2M} \cos \frac{(2y+1)n\pi}{2N} *$$

*<https://www.mathworks.com/help/vision/ref/2ddct.html>

2.4 Transformata Fourier discretă

Transformata Fourier discretă este folosită în procesarea audio pentru a transforma un semnal discret (eșantionat la o frecvență aleasă) din domeniul timp în domeniul frecvență. Rezultatul aplicării transformatei Fourier peste un vector de valori reale este un alt vector de valori complexe, numite coeficienți Fourier. Coeficienții obținuți conțin informație referitor la mărimea (modulul numerelor complexe) și faza (unghiul dintre componenta reală și cea imaginară a unui număr pe planul complex).

Formula pentru transformata Fourier folosită în cod este următoarea:

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-i2\pi \frac{k}{N}n}$$

2.5 Interpretarea unui semnal audio digital

Pentru a înțelege modul în care un semnal audio este stocat și procesat de un calculator trebuie să înțelegem următoarele: semnalul audio este digital, însemnând că fidelitatea față de sunetul real se poate realiza până la un anumit punct; Spectrul uman audio poate deduce frecvențe între 20 Hz și 20 kHz; Conform teoremei de eșantionare Nyquist-Shannon, pentru a putea reproduce frecvența de 20 kHz fără a avea aliere, trebuie să eșantionăm un semnal audio analog cu o frecvență de cel puțin 40 kHz; în mod uzual, frecvența de eșantionare pentru semnale audio digitale este de 44.1 kHz, convenția fiind stabilită de conglomeratul Sony în anul 1979¹. Așadar, pentru fiecare secundă a unei înregistrări audio, avem 44100 de eșantioane.

[†]<https://docs.scipy.org/doc/scipy/reference/generated/scipy.fft.fft.html>

¹https://en.wikipedia.org/wiki/44,100_Hz

Capitolul 3

Implementări

3.1 Despre implementare

Codul folosit pentru documentarea lucrării este realizat în limbajul de programare Python și este accesibil pe pagina [aceasta](#) de GitHub.

Pe lângă biblioteca standard Python am folosit de următoarele biblioteci: `PIL.Image` pentru citirea și salvarea de imagini, `numpy` pentru operații matematice și procesarea paralelizată a instrucțiunilor, `matplotlib` pentru prezentarea graficelor și imaginilor, `scipy` pentru operații matematice complexe (transformata Fourier, convoluție, transformata cosinus discretă) și citirea fișierelor audio, `bitstring` pentru a simplifica operațiile de conversie din baza hexazecimală în baza binară, `torch` pentru implementarea rețelei neurale și `jpegio` pentru accesarea coeficienților DCT pentru imagini jpg.

3.2 Metrici de măsurare

Folosind metricile de măsurare prezentate în continuare, putem deduce matematic nivelul de degradare pe care un model aplicat îl produce mediului. Aceste metrici sunt folosite pentru a analiza gradul de imperceptibilitate pe care un model îl oferă. În mod uzual se compară toate cele trei metrici, deoarece fiecare oferă o interpretare distinctă a nivelului de degradare. Toate metodele de mai jos se bazează pe existența imaginii originale pentru a efectua calculele. Metricile pot fi folosite pentru a analiza imperceptibilitatea, dar nu pot fi folosite în steganaliză (pot crea un mediu de ascundere nou, să ascund mesajul, după care să trimit mai departe doar conținutul modificat). În formulele de mai jos, I și J reprezintă imaginea nealterată, respectiv imaginea stego.

3.2.1 Eroarea pătratică medie

Formula pentru *Eroarea pătratică medie* (Mean Squared Error - MSE) este următoarea:

$$MSE(I, J) = \sum_{i=1}^w \sum_{j=1}^h [I(i, j) - J(i, j)]^2$$

3.2.2 Raportul semnal maxim-zgomot

Raportul semnal maxim-zgomot (Peak Signal-To-Noise Ratio - PSNR) este o metodă folosită în mod tradițional pentru a măsura gradul până la care un semnal a fost degradat în urma unui proces (blurare, compresie, filtrare sau în cazul de față ascunderea mesajului). Rezultatul metricii este exprimat în decibeli, o valoare mică (20-25 dB) reprezentând degradare mare, iar o valoare mare (60 dB) reprezentând degradare minimă.

Formula pentru PSNR este următoarea:

$$PSNR(I, J) = 10 \log_{10} \left(\frac{255^2}{(MSE(I, J))} \right)$$

3.2.3 Indicele de similaritate structurală mediu

Indicele de similaritate structurală mediu (Mean Structured Similarity Indexing Method - MSSIM) a fost introdus în 2004 de Wang et al.[5] ca o alternativă la cele două metrici prezentate anterior. Diferența este dată de faptul că MSSIM ia în calcul percepția vizuală a omului (celelalte 2 metrici se folosesc doar de valorile semnalului, ori MSSIM se bazează pe eroarea structurală a imaginii). Valorile sunt între 0 și 1, 0 însemnând degradare maximă, iar 1 similaritate perfectă.

Formulele pentru SSIM/MSSIM sunt următoarele:

$$SSIM(I, J) = \frac{(2\mu_I\mu_J + C_1)(2\sigma_{IJ} + C_2)}{(\mu_I^2 + \mu_J^2 + C_1)(\sigma_I^2 + \sigma_J^2 + C_2)},$$

$$\mu_I = \sum_{i=1}^N w_i I_i,$$

$$\sigma_I = \sqrt{\sum_{i=1}^N w_i (I_i - \mu_I)^2} = \sqrt{\sum_{i=1}^N w_i I_i^2 - \mu_I^2},$$

$$\sigma_{IJ} = \sum_{i=1}^N w_i (I_i - \mu_I)(J_i - \mu_J),$$

$$MSSIM(I, J) = \frac{1}{M} \sum_{j=1}^M SSIM(I_j, J_j)$$

Formulele de sus pot fi înțelese mai ușor având în vedere următoarele:

- w_i reprezintă o matrice cu 11 linii și 11 coloane, mai exact un filtru Gaussian cu

$\sigma = 1.5$ și suma elementelor egală cu 1, iar I_i reprezintă o matrice cu 11 linii și 11 coloane din imaginea I .

- μ_I poate fi calculat aplicând filtrul Gaussian peste imaginea I folosind operația de convoluție.
- σ_I poate fi calculat bazându-ne pe formula deviației standard: $\sigma = \sqrt{E[(X - \mu)^2]}$. În cazul nostru, pentru că $\sum_{i=1}^N w_i = 1$, termenul din mijloc din desfășurarea diferenței pătratice este $-2 \times \mu_I \times \sum_{i=1}^N w_i I_i = -2 \times \mu_I^2$. Acesta se reduce cu termenul din dreapta ($\mu_I^2 \times \sum_{i=1}^N w_i$).
- MSSIM este media matricelor obținute anterior (considerăm că avem 3 canale de culoare, fiecare având ca rezultat propria lui matrice SSIM).
- C_1 și C_2 sunt valori constante, $C_1 = (0.01 \times 255)^2$, iar $C_2 = (0.03 \times 255)^2$

3.3 Modele de steganografie

3.3.1 Metoda bitului cel mai puțin semnificativ

Ideea de bază a metodei *Metoda bitului cel mai puțin semnificativ* (Least Significant Bit - LSB) este modul în care alterarea bitului cel mai puțin semnificativ (cel mai din dreapta în reprezentarea binară a valorii unui pixel) afectează imaginea. Privite cu ochiul liber, diferențele între imaginea originală și cea alterată sunt mici. Deoarece fiecare pixel al imaginii este alterat, metoda oferă o capacitate de ascundere ridicată. Să luăm drept exemplu o imagine color de dimensiune 512×512. Dacă alegem primul LSB ca spațiu de ascundere, vom obține o capacitate de 786432 biți.

Implementarea prezentată poate fi particularizată pentru a lua mai mulți biți din reprezentarea binară drept spațiu de ascundere. Parcurgerea se face mai întâi peste fiecare canal de culoare al unui pixel, accesând cei k biți selectați drept spațiu de ascundere și parcurgerea pixelilor de la stânga la dreapta, de sus în jos.



Figura 3.1: Diferența între imaginea originală și cea stego, model LSB

3.3.2 Diferențierea valorilor pixelilor

Propusă de Wu et al.[6] în anul 2003, metoda *diferențierii valorilor pixelilor* (Pixel Value Differencing - PVD) presupune ascunderea mesajului în spațiul imaginii, pornind de la următoarea observație: zonele din imagine unde nu sunt prezente schimbări de culoare au diferențe mici între intensitatea pixelilor vecini, iar zonele în care apar muchii sau schimbări bruște de culoare au diferențe mai mari. Cunoscând această informație, metoda Pixel Value Differencing propune ascunderea unui mesaj în imagine, modificarea esențială fiind faptul că numărul de biți modificați variază în funcție de diferența pixelilor adiacenți.

Capacitatea de ascundere poate fi particularizată pentru fiecare treaptă de diferențe după o listă de cuantizare Q . Lista trebuie să aibă elemente cu suma totală egală cu 256. Fiecare element k reprezintă numărul de elemente al unui interval, capacitatea de ascundere al respectivului interval fiind $c = \log_2(k)$.

În continuare va fi prezentat un exemplu didactic. Fie șirul $m = 110$ o subsecvență din mesajul de ascundere, $x = [101, 103, 113, 121]$ două perechi de pixeli adiacenți și $Q = [1, 1, 2, 4, 4, 4, 8, 8, 16, 16, 32, 32, 64, 64]$.

Pentru Q vom obține intervalele: $[[0, 1), [1, 2), [2, 4), [4, 8), [8, 12), [12, 16), [16, 24), [24, 32), [32, 48), [48, 64), [64, 96), [96, 128), [128, 192), [192, 256]]$. $\Delta_1 = |101 - 103| = 2$ și $\Delta_2 = |113 - 121| = 8$. Δ_1 pică în intervalul $[2, 4)$, iar Δ_2 pică în intervalul $[8, 12)$. Capacitatea aferentă primului interval este de 1 bit, iar cea aferentă celui de-al doilea interval este de 2 biți. Din m voi lua un bit (1), respectiv 2 biți (10) pe care să îi incorporez. Așadar, Δ'_1 va trebui să fie 3 ($2 + 1$), iar Δ'_2 va trebui să fie 10 ($8 + 2$). Δ' Este calculat luând reprezentarea în baza 10 a valorii biților pe care vrem să îi ascundem, la care adunăm extremitatea inferioară a intervalului de capacitate. Perechea x'_1 va deveni $[101, 104]$, iar perechea x'_2 va deveni $[112, 122]$.

Algoritmul de ascundere parcurge imaginea în zig-zag (de la stânga la dreapta pentru

o linie și viceversa pentru următoarea linie), pentru fiecare componentă de culoare. Algoritmul de extragere face aceeași parcurgere, iar dată fiind lista de cuantizare folosită, se poate deduce în ce interval pică o diferență între pixeli adiacenți, deci totodată care este subsecvența din mesaj care poate fi extrasă din acea pereche.

3.3.3 Transformata cosinus discretă

Pentru modelul bazat pe *Transformata cosinus discretă* (Discrete Cosine Transform - DCT) am implementat două metode: prima are ca reper prezentarea făcută de Gibson[1], în care pentru o imagine în format png parcurg toți pașii pentru compresie jpeg, dar care îmi returnează pe lângă imaginea stego și matricea de coeficienți DCT pentru componenta de culoare Y (aceasta se întâmplă deoarece, în procesul de reconstruire al unei imagini din informațiile comprimate, intensitățile pixelilor obținuți ies din intervalul $[0, 255]$, fiind necesară o rotunjire și o limitare). A doua metodă modifică direct coeficienții unei imagini jpeg salvată în memorie. Pentru a fi mai ușor de urmărit, mă voi referi în continuare la prima implementare drept DCT, iar la cea de a doua drept DCT_ jpeg.

Variantă PNG

Metoda aceasta creează propria matrice de coeficienți, urmând pașii unui pipeline de compresie jpeg:

- Citesc imaginea în format RGB
- Fac padding la o dimensiune multiplu de 16 (după media pe Cb și Cr vreau dimensiune multiplu de 8)
- Fac conversie din RGB în YCbCr
 - Pentru canalele Cb și Cr, fac medie pentru patch-uri de dimensiune 2x2
- Aplic DCT pentru patch-uri de 8x8 din toate cele 3 canale de culoare
- Împart matricea aferentă componentei Y cu o matrice `Q_luminance`, iar componentele Cb și Cr cu o altă matrice `Q_chrominance`
- Ascund mesajul în matricea de coeficienți pentru componenta Y.

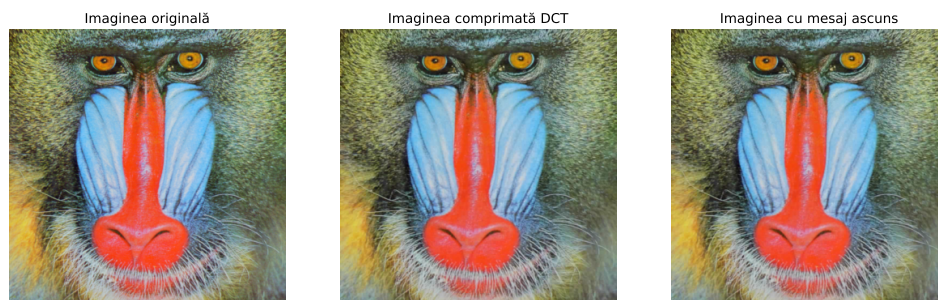


Figura 3.2: Prezentarea rezultatelor obținute folosind metoda DCT pentru o imagine curată și o imagine stego

Metoda aceasta prezintă interes deoarece alterarea unui coeficient nu duce la modificarea unui singur pixel, ci a mai multor pixeli (o componentă schimbată în domeniul frecvență poate duce la mai multe schimbări în domeniul spațiu).

Varianța JPEG



Figura 3.3: Prezentarea rezultatelor obținute folosind metoda DCT_jpeg pentru o imagine curată și o imagine stego

3.3.4 Codarea de fază

Codarea de fază (Phase Encoding – PE) presupune împărțirea unui fișier audio în segmente de dimensiune egală (lungime putere a lui 2, iar dacă asta nu se întâmplă în mod natural, fac padding la final cu 0, până când lungimea este multiplu de o putere a lui 2), aplicarea transformatei Fourier rapide peste fiecare segment, obținerea vectorilor de magnitudini și de faze, urmat de ascunderea mesajului în vectorul de faze obținut.

Spectrul de frecvență obținut aplicând transformata Fourier este simetric conjugat, drept pentru care prima jumătate a vectorului conține frecvențele pozitive, pornind de la

0, iar a doua jumătate conține frecvențele negative. Din acest considerent, pentru a păstra simetria în momentul aplicării transformatei Fourier inverse, atunci când modific o fază din segmentul meu, am grijă să fac aceeași modificare în cealaltă jumătate a spectrului, dar cu semn opus.

Fazele reprezintă valori exprimate în radiani. Pentru un bit 0 din mesajul meu setez faza la $\frac{\pi}{2}$ iar pentru un bit 1 setez faza la $-\frac{\pi}{2}$.

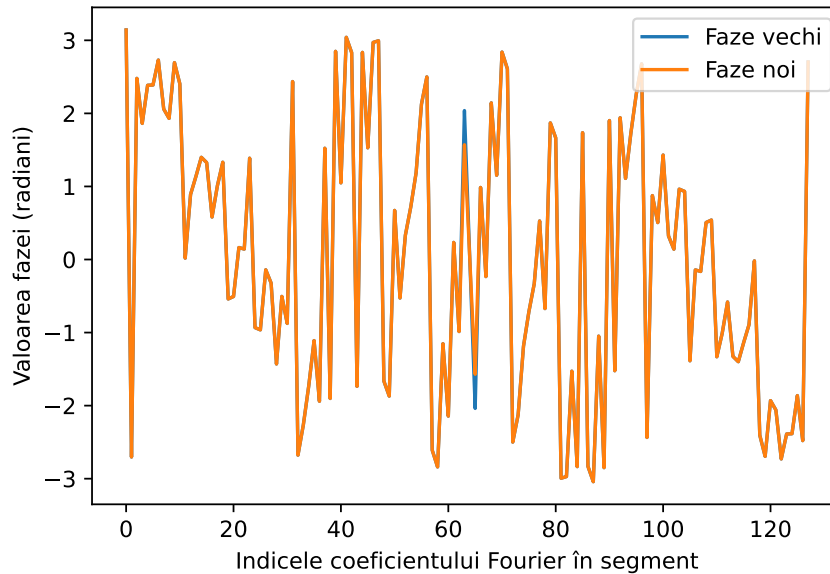


Figura 3.4: Diferența între fazele inițiale pentru un segment și fazele finale pentru un segment

Extragerea se face împărțind fișierul audio modificat în segmente de lungime identică, după care se aplică transformata Fourier, obțin vectorul de faze și verific semnul fazei modificate în segment (dacă e pozitiv, atunci am un bit 0, altfel am un bit 1).

O îmbunătățire făcută față de metoda prezentată de Yang[7] în ceea ce privește rata de eroare o reprezintă includerea unei singure modificări în vectorul de faze al unui segment, pentru fiecare segment, în acest mod îmbunătățind recuperabilitatea mesajului transmis. Astfel, pot obține recuperabilitate mare și pentru dimensiuni de segment mici, făcând un compromis în ceea ce privește imperceptibilitatea.

3.4 Steganaliză

3.4.1 Inspecție vizuală

Un exemplu de inspecție vizuală este inspecția per-bit al unui canal culoare al imaginii. Practic metoda se rezumă la aplicarea unei măști de biți peste fiecare pixel din imagine și normalizarea în intervalul de valori $[0, 255]$. Metoda funcționează destul de bine pentru

detectarea conținutului ascuns folosind modelul LSB.

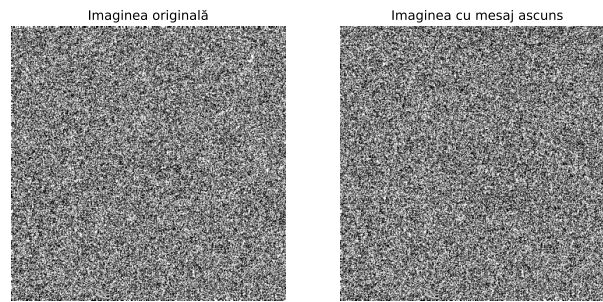


Figura 3.5: Inspecție vizuală pentru imaginea de test Lenna a canalului de culoare R, model LSB

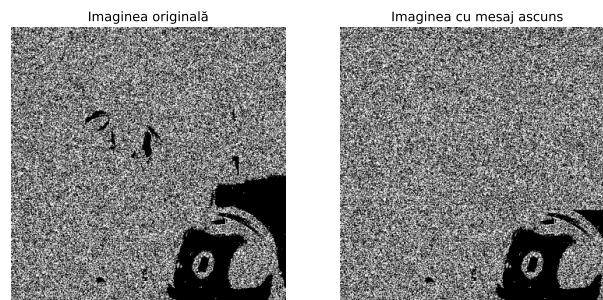


Figura 3.6: Inspecție vizuală pentru imaginea de test Astronaut a canalului de culoare R, model LSB, a se observa zonele continue care sunt înlocuite

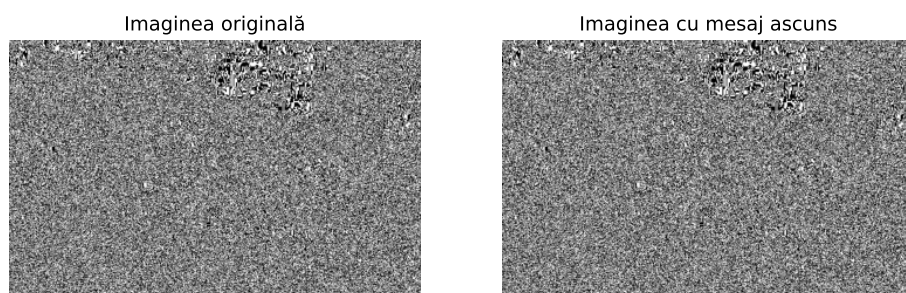


Figura 3.7: Inspecție vizuală a canalului de culoare G, mască pe 2-LSB, model PVD, a se observa că modelul păstrează zonele continue

Pe lângă faptul că aceste metode de inspecție vizuală ar presupune ca o persoană să observe fiecare imagine pentru a trage concluzii (ceea ce în mod firesc nu este optim), nu putem să ne bazăm întotdeauna că vom putea găsi diferențe.

3.4.2 Metode statistice

Una dintre metodele statistice pe care le putem aplica este calculul distanței între două distribuții. Obținerea distribuțiilor depinde de la model la model.

Lin et al.[3] prezintă o metodă de a oferi un scor pe baza căruia să clasificăm o imagine ca fiind alterată folosind modelul PVD. Distribuția în acest caz reprezintă numărul de apariții al unei diferențe între valorile pixelilor adiacenți. Luând intervalul $[-7, 7]$ (din intervalul total posibil $[-255, 255]$) și aducându-l într-un spațiu probabilistic (de sumă 1), pentru o mulțime de 2500 de imagini curate, obținem următoarea distribuție:

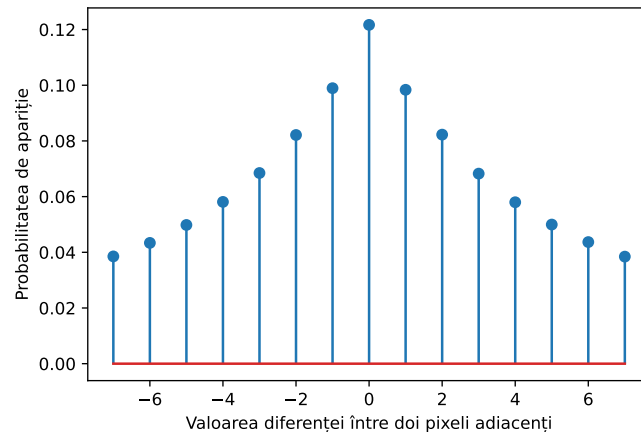


Figura 3.8: Distribuția diferențelor între pixeli adiacenți într-o imagine curată

Pentru a avea un termen de comparație, pe lângă metrica implementată de Lin et al., am decis să implementez și o metodă de scor bazată pe divergența Kullback-Leibler:

$$D_{KL}(P||Q) = \sum_{x \in X} P(x) \log \frac{P(x)}{Q(x)}^{\dagger}$$

unde P este distribuția pe care mă aștept să o am, iar Q este distribuția reală.

Folosindu-mă de aceeași distribuție de probabilități de mai sus, am trecut pozele din mulțimea PVD_stego pentru a vedea dacă metoda reduce numărul de fals-pozitive. Într-adevăr, divergența Kullback-Leibler delimitează mai bine cele două mulțimi.

[†]https://en.wikipedia.org/wiki/Kullback%E2%80%93Leibler_divergence

3.4.3 Clasificator bazat pe rețele neurale

Am implementat o rețea neurală pentru a clasifica dacă o imagine este curată sau nu pentru modelul DCT_jpeg. Datele folosite sunt distribuțiile obținute din coeficienții prezenți într-o imagine (altfel spus, fiecare caracteristică reprezintă numărul de apariții al valorii drept coeficient în matricea de coeficienți). Valorile coeficienților sunt în intervalul $[-1024, 1024]$, obținând în total 2049 de caracteristici.

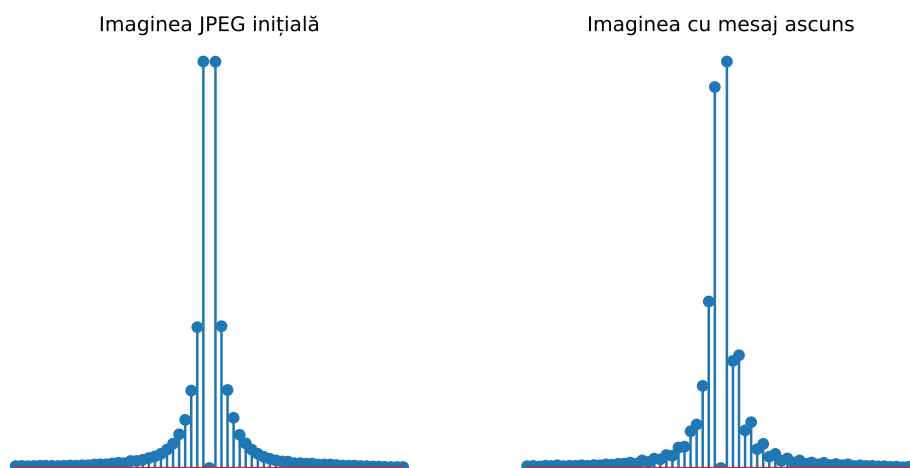


Figura 3.9: Reprezentare a vectorilor de intrare pentru rețeaua neurală (din motive de afișare, coeficientul 0 a fost setat la zero, deși în realitate numărul de apariții este exponențial mai mare)

Mulțimea de date a fost obținută aplicând algoritmul DCT_jpeg peste un set de 75000 de poze curate, din dataset-ul ALASKA2¹. Am extras distribuțiile din setul de imagini original și cel prelucrat, după care am antrenat modelul, obținând din aceste date un F1 score de 0.9993 (pe mulțimea de validare, split-ul folosit fiind 60-20-20). Pentru modele mai avansate, precum UERD (Guo et al. 2015[2]), clasificatorul nu mai obține aceleași rezultate.

3.5 Codificare Huffman - Crearea mesajului de ascuns

Ideea de a implementa coduri Huffman în lucrarea de licență a venit citind lucrarea lui Sanjalawe et al.[4].

Una dintre principalele probleme descoperite în implementările propuse anterior este cea a spațiului de ascundere pe care fiecare metodă îl oferă. În funcție de tipul mesajului

¹<https://www.kaggle.com/competitions/alaska2-image-steganalysis/data>

pe care vrem să îl ascundem, putem să îi reducem dimensiunea folosind coduri Huffman.

Am făcut implementarea deoarece am vrut să demonstrez cum pot ascunde un fișier executabil într-un mediu. Deoarece fișierele executabile au foarte mult spațiu gol, codurile Huffman sunt foarte eficiente în a reduce dimensiunea. Implementarea este particularizată să accepte dimensiuni variabile pentru fiecare caracter.

Alături de compresie, am creat un mecanism prin care pot plasa metadate despre unde este zona de mesaj comprimat și unde începe zona care descrie tabela de recuperare pentru coduri Huffman. În cod, clasa **MessageParser** face tocmai asta:

- Preia mesajul bitstring original și îl comprimă, întorcând mesajul comprimat alături de tabela de recuperare (în cazul meu un bitstring care encodează o parcurgere DFS a grafului generat de algoritmul de compresie Huffman)
- Stochează dimensiunea mesajului pe 32 de biți
- Stochează dimensiunea alfabetului folosit de tabela Huffman pe 8 biți
- Fac padding până toate valorile anterioare adunate sunt multiplu de 8 (padding-ul se face cu negația MSB-ului din mesaj, adică pentru un șir $m = 0100\dots$, varianta padded va fi de forma $m' = 110100\dots$).
- Iau lungimea șirului obținut din concatenarea padding-ului, dimensiunii mesajului comprimat, dimensiunea alfabetului, mesajul comprimat și tabela Huffman.
- Concatenez lungimea obținută anterior.

Cu aceste metadate pot face o singură extragere peste întreg spațiul de ascundere pentru un model dat, după care să extrag doar informațiile de interes fără a specifica care este lungimea (cu alte cuvinte, nu mai trebuie să știu a priori care este lungimea mesajului care urmează a fi extras, este suficient doar să am algoritmul de parsat un mesaj).

Capitolul 4

Rezultate

În continuare vor fi prezentate rezultatele obținute de modelele propuse pentru o imagine de test și un fișier audio.

4.1 Scoruri obținute

Model	MSE	PSNR	MSSIM	BPP
1-LSB	0.499	51.14	99.55%	3.00
2-LSB	2.522	44.11	97.81%	6.00
PVD_q1	1.206	47.31	99.72%	1.71
PVD_q2	5.780	40.51	97.37%	4.66
DCT	6.144	40.13	97.93%	0.05
DCT_jpeg	6.616	39.92	97.96%	0.11
PE_128	1918.83	63.49	99.59%	na
PE_256	506.80	69.28	99.98%	na

Tabela 4.1: Scoruri obținute de modelele implementate, scorurile pot diferi în funcție de lungimea mesajului

Mediu	Dimensiune (kB)
PNG	503.8
JPEG	38.7
WAV	490.8

Tabela 4.2: Mediile folosite pentru testare și dimensiunea lor pe disc

Model	Ascundere	Extragere
1-LSB	2.603	0.314
2-LSB	2.561	0.319
PVD_q1	5.238	1.285
PVD_q2	12.862	1.818
DCT	1.146	0,131
DCT_jpeg	0.337	0.189
PE_128	0.014	0.004
PE_256	0.013	0.004

Tabela 4.3: Durata (exprimată în secunde) pentru ascundere și extragere

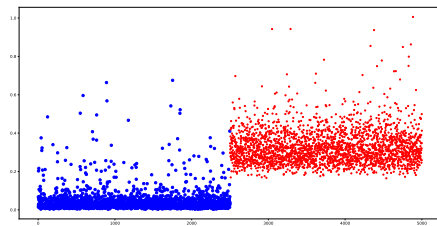
Model	Capacitate	BER
PE_128	1918	3.023%
PE_256	959	2.614%

Tabela 4.4: Rata de eroare și capacitatea (exprimată în biți) dată de modelele Phase Encoding

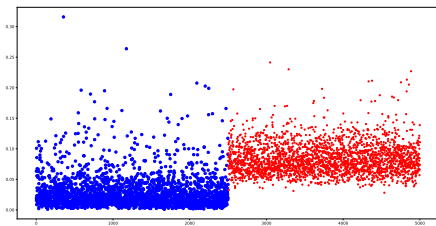
Lungimea mesajului folosit este egală cu capacitatea maximă ce poate fi ascunsă folosind fiecare model propus. Pentru PVD, listele de cuantizare folosite sunt $q1 = [1, 1, 2, 4, 4, 4, 8, 8, 16, 16, 32, 32, 64, 64]$, $q2 = [8, 8, 16, 32, 64, 128]$.

4.2 Scoruri folosind metode statistice

În plot-urile ce vor urma, punctele albastre reprezintă scorurile obținute de imaginile fără conținut ascuns, iar cele roșii scorurile obținute de imaginile stego.

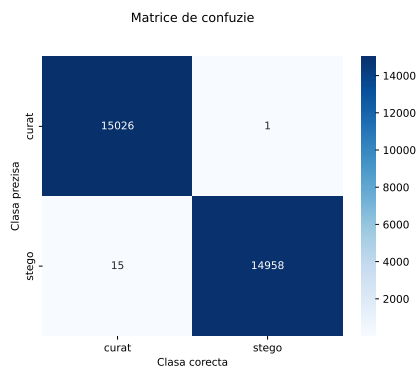


(a) Scorurile dintre distribuții pentru imagini curate și imagini stego folosind divergența Kullback-Leibler

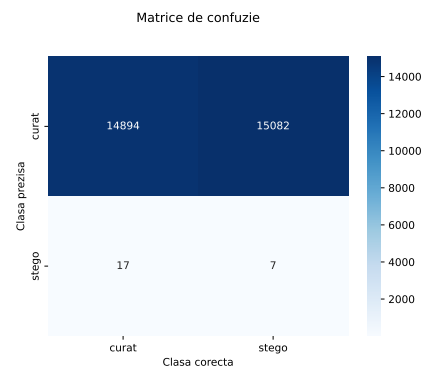


(b) Scorurile dintre distribuții pentru imagini curate și imagini stego folosind scorul propus de Lin et al.

4.3 Matrice de confuzie pentru rețeaua neurală



(a) Matrice de confuzie pentru mulțimea de test obținută de către rețeaua neurală pentru DCT_jpeg și imagini curate



(b) Matrice de confuzie pentru mulțimea de test obținută de către rețeaua neurală pentru UERD și imagini curate

Capitolul 5

Concluzii

Raportându-ne la cifrele obținute anterior, putem deduce unde se descurcă cel mai bine modelele.

Modelul LSB oferă cel mai bun raport dintre capacitatea de ascundere și pierderile observate (3 biți per pixel pentru o valoare MSSIM de 99,55%). Timpii de ascundere și de extragere sunt un compromis între PVD și DCT.

Modelul PVD cu lista de cuantizare $q1$ furnizează cel mai bun scor MSSIM dintre metodele testate pe imagini, de 99.72% (și are sens, pentru că zonele în care pixelii adiacenți au diferență mică nu suferă modificări, păstrând integritatea structurală). Compromisul major însă este la timpul mare necesar pentru ascundere și extragere, alături de pierderea înregistrată în capacitatea de ascundere.

Modelul DCT (oricare dintre variante) a obținut cele mai slabe scoruri dintre modelele comparate. Singurul avantaj este timpul mic pentru ascundere și extragere.

Modelul PE are un MSE mult mai mare față de alte modele testate, deoarece valorile cu care lucrează un semnal audio se află în alt interval valoric (imaginile sunt în intervalul $[0, 255]$, semnalul audio în intervalul $[-32.768, 32.767]$), PSNR și MSSIM sunt cele mai bune dintre toate modelele testate. O pierdere mare însă este pierderea de robustețe pentru un mesaj foarte lung, alături de capacitatea de ascundere mică raportată la dimensiunea fișierului audio.

5.1 Direcții viitoare de cercetare

- Explorarea unor metode mai avansate de ascundere (JUNIWARD, SteganoGAN).
- Centralizarea studiilor într-o aplicație care poate face detecție / filtrare în timp real.
- Explorarea altor modele mai robuste (la atac cu zgomot sau compresie, precum F5).

Bibliografie

- [1] Ryan Gibson, *Steganography: Hiding Data In Plain Sight*, <http://www.cs.unc.edu/~lin/COMP089H/LEC/steganography.pdf>, Department of Computer Science, College of Arts and Sciences, The University of North Carolina at Chapel Hill, 2015.
- [2] Linjie Guo, Jiangqun Ni, Wenkang Su, Chengpei Tang și Yun-Qing Shi, „Using Statistical Image Model for JPEG Steganography: Uniform Embedding Revisited”, în *IEEE Transactions on Information Forensics and Security* 10.12 (2015), pp. 2669–2680, DOI: [10.1109/TIFS.2015.2473815](https://doi.org/10.1109/TIFS.2015.2473815).
- [3] Wen-Bin Lin, Tai-Hung Lai și Ko-Chin Chang, „Statistical Feature-Based Steganalysis for Pixel-Value Differencing Steganography”, în *EURASIP Journal on Advances in Signal Processing* 2021.1 (2021), p. 87, DOI: [10.1186/s13634-021-00797-5](https://doi.org/10.1186/s13634-021-00797-5).
- [4] Yousef Sanjalawe, Salam Al-E'mari, Salam Fraihat, Mosleh Abualhaj și Emran Alzubi, „A Deep Learning-Driven Multi-Layered Steganographic Approach for Enhanced Data Security”, în *Scientific Reports* 15.1 (2025), p. 89189, DOI: [10.1038/s41598-025-89189-5](https://doi.org/10.1038/s41598-025-89189-5).
- [5] Zhou Wang, Alan C Bovik, Hamid R Sheikh și Eero P Simoncelli, „Image quality assessment: From error visibility to structural similarity”, în *IEEE Transactions on Image Processing* 13.4 (2004), pp. 600–612, DOI: [10.1109/TIP.2003.819861](https://doi.org/10.1109/TIP.2003.819861).
- [6] Da-Chun Wu și Wen-Hsiang Tsai, „A steganographic method for images by pixel-value differencing”, în *Pattern Recognition Letters* 24.9-10 (2003), pp. 1613–1626, DOI: [10.1016/S0167-8655\(02\)00402-6](https://doi.org/10.1016/S0167-8655(02)00402-6).
- [7] Guang Yang, *An Improved Phase Coding Audio Steganography Algorithm*, <https://arxiv.org/abs/2408.13277>, arXiv:2408.13277 [cs.CR], version 2, 27 Aug 2024, 2024.